



Smart Transit Companion for Indian State Transport

"Where Is My Bus" — A real-time, GPS-powered, multi-modal transit companion for Indian state transport passengers, inclusive of elderly & feature-phone users through an AI-powered IVR system.

Implementation Plan & Technical Architecture

Smart India Hackathon 2026

Team: Beyond Binary

Date: August 2026

Version: 1.0

Classification: Team Internal — Confidential

1. Problem Statement & Context

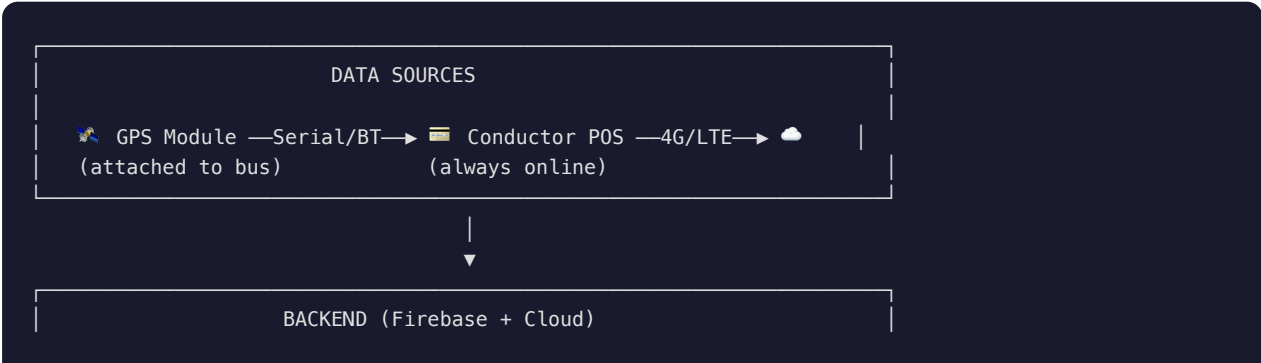
Indian state transport corporations (MSRTC, KSRTC, UPSRTC, APSRTC, etc.) carry **~70 million passengers daily**, yet passengers have:

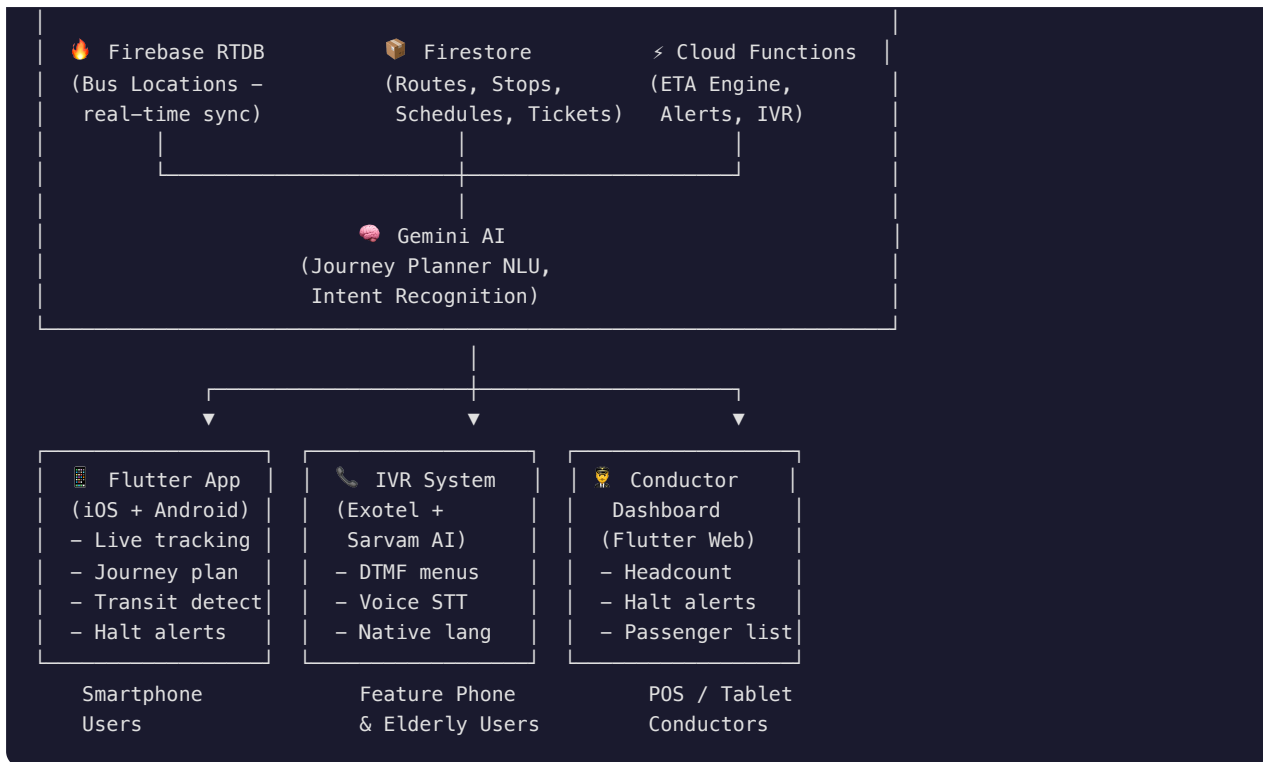
- **Zero real-time visibility** into bus locations on most routes
- **No reliable multi-bus journey planning** (unlike "Where is my Train" for railways)
- **No accessibility** for the elderly or feature-phone users (~300M+ users in India)
- **No night-halt alerts** — passengers get stranded at food stops
- **Conductors lack digital headcount** tools for halt-stop management

1.1 What Exists vs. What We Build

Feature	Existing Solutions ("Aapli ST", etc.)	BussPass (Ours)
GPS Tracking	✅ Basic (16K buses)	✅ POS-powered GPS (leverages existing infrastructure)
Multi-bus Journey Planning	❌ Not available	✅ Graph-based route planner with real-time ETA
Smart Transit Detection	❌ Not available	✅ Auto-detect if user is inside a bus
Connection Bus Management	❌ Not available	✅ Real-time connecting bus availability
Feature Phone / Elderly Support	❌ Not available	✅ IVR + DTMF + Native language STT
Night Halt Alerts	❌ Not available	✅ Push + automated phone call alerts
Conductor Headcount Dashboard	❌ Not available	✅ Digital passenger manifest

2. System Architecture Overview





3. Technology Stack

3.1 Mobile App (Passengers + Conductor)

Layer	Technology	Justification
Framework	Flutter 3.x (Dart)	Single codebase → iOS + Android + Web
State Management	Riverpod 2.x	Reactive, testable, clean architecture
Maps	Google Maps Flutter	Best India coverage, traffic layer, polylines
Location	geolocator + flutter_background_service	Background location for transit detection
Notifications	Firebase Cloud Messaging (FCM)	Push alerts for halt stops, ETA reminders
Local DB	Hive or Isar	Offline route cache for low-connectivity areas

3.2 Backend

Layer	Technology	Justification
Real-time DB	Firebase Realtime Database	Ultra-low latency (~200ms) for live bus positions
Persistent DB	Cloud Firestore	Routes, stops, schedules, tickets, user journeys

Serverless	Firestore Cloud Functions (TypeScript)	ETA calculations, alerts, IVR webhooks
AI/ML	Gemini 2.5 Flash (Firestore AI Logic)	Journey planning NLU, IVR intent recognition
Auth	Firestore Auth	Phone OTP (works with all Indian numbers)
Hosting	Firestore Hosting	Admin dashboard, conductor web app

3.3 IVR System

Layer	Technology	Justification
Telephony	Exotel	PSTN-compatible (feature phones), Indian compliance
Speech-to-Text	Sarvam AI (Saarika)	10+ Indian languages, handles code-switching
Text-to-Speech	Sarvam AI (Bulbul)	Natural Indian language voices
NLU/Intent	Gemini 2.5 Flash	Parse user intent from transcribed text

3.4 GPS + POS Integration

Component	Model	Purpose
GPS Module	u-blox NEO-6M	Satellite positioning
Bluetooth Module	HC-05	GPS → POS wireless link
Voltage Regulator	LM7805 (12V→5V)	Power from bus battery
Enclosure	Generic ABS box	Weatherproof mounting

4. Feature Modules — Detailed Design

4.1 🌍 Real-Time Bus Tracking ("Where is my Bus")

Firestore RTDB Schema

```
{
  "buses": {
    "MH12AB1234": {
      "lat": 18.5204,
      "lng": 73.8567,
      "speed": 45.2,
      "heading": 180,
      "routeId": "PUNE-MUM-001",
```

```

    "nextStopId": "LONAVALA",
    "etaNextStop": 1724700000,
    "status": "IN_TRANSIT",
    "lastUpdated": 1724699500,
    "passengerCount": 38
  }
}
}

```

How It Works

1. GPS module on bus captures lat/lng every 10 seconds
2. POS background app reads GPS data via Bluetooth/Serial
3. App pushes `{lat, lng, speed, heading, timestamp}` to `buses/{busId}` in RTDB
4. Flutter app subscribes via `StreamBuilder` → map marker moves in real-time
5. Cloud Function calculates ETA to next stops using speed + route geometry

Data Consumption

- 1 update ≈ 200 bytes JSON
- 6 updates/min × 60 min × 16 hrs = 5,760 updates/day
- 5,760 × 200 bytes = **~1.15 MB/day per bus** — trivial on any data plan

4.2 Multi-Bus Journey Planner

Core Algorithm: Modified Dijkstra on Transit Graph

Input: Source Stop, Destination Stop, Departure Time
 Output: Ordered list of [Bus, Board Stop, Alight Stop, ETA]

Algorithm:

1. Build weighted directed graph:
 - Nodes = All bus stops
 - Edges = Direct bus connections (weight = travel time)
 - Transfer edges = Walking between nearby stops (weight = walk time + estimated wait time)
2. Run time-dependent Dijkstra:
 - At each node, check which buses are AVAILABLE at that time
 - Query RTDB for real-time bus positions for actual ETAs
 - Factor in buffer time for connections (minimum 10 min)
3. Return top 3 routes ranked by:
 - Total travel time
 - Number of changes required
 - Reliability score (based on bus punctuality data)

Firestore Route Schema

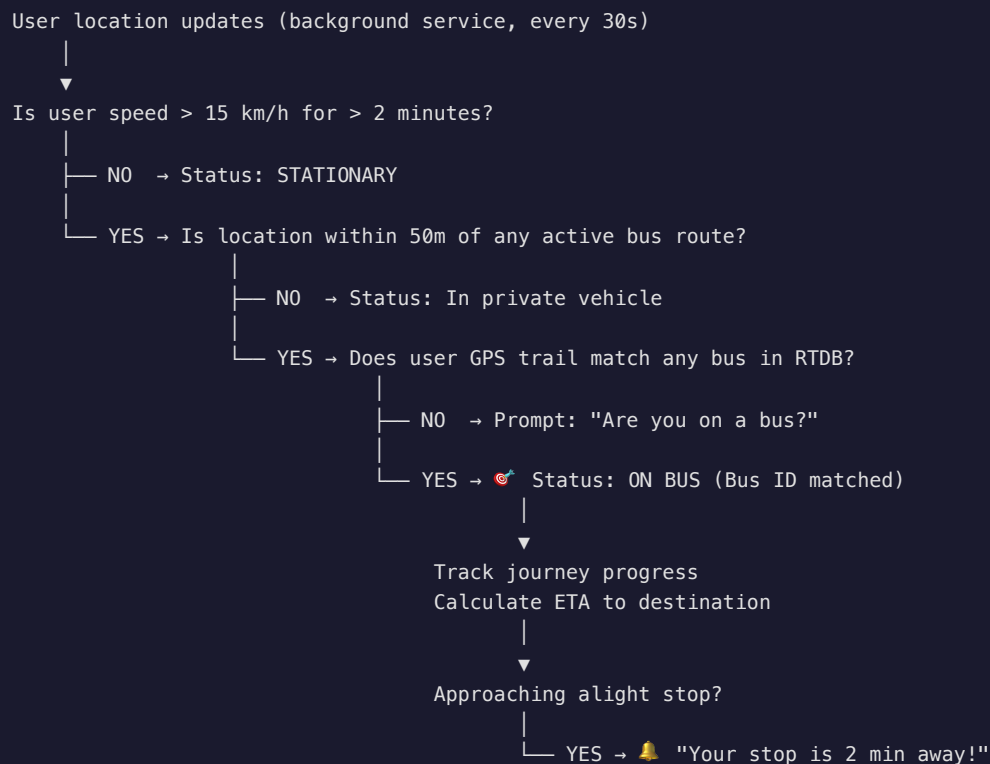
```

{
  "routes/PUNE-MUM-001": {
    "name": "Pune → Mumbai (via Expressway)",
    "operator": "MSRTC",
    "stops": [
      {"id": "SWARGATE", "name": "Swargate", "lat": 18.5018, "lng": 73.8636, "seq": 1},
      {"id": "LONAVALA", "name": "Lonavala", "lat": 18.7546, "lng": 73.4062, "seq": 3},
      {"id": "DADAR", "name": "Dadar", "lat": 19.0178, "lng": 72.8478, "seq": 8}
    ],
    "schedule": {
      "frequency": "every 30 min",
      "firstBus": "05:00",
      "lastBus": "23:30"
    },
    "avgDuration": 240
  }
}

```

4.3 📍 Smart Transit Detection

Detection Logic Flow



Key Technique: Cross-reference user's GPS trail with bus GPS trail from RTDB. If both are moving along the same route with < 100m deviation for > 3 minutes → user is on that bus.

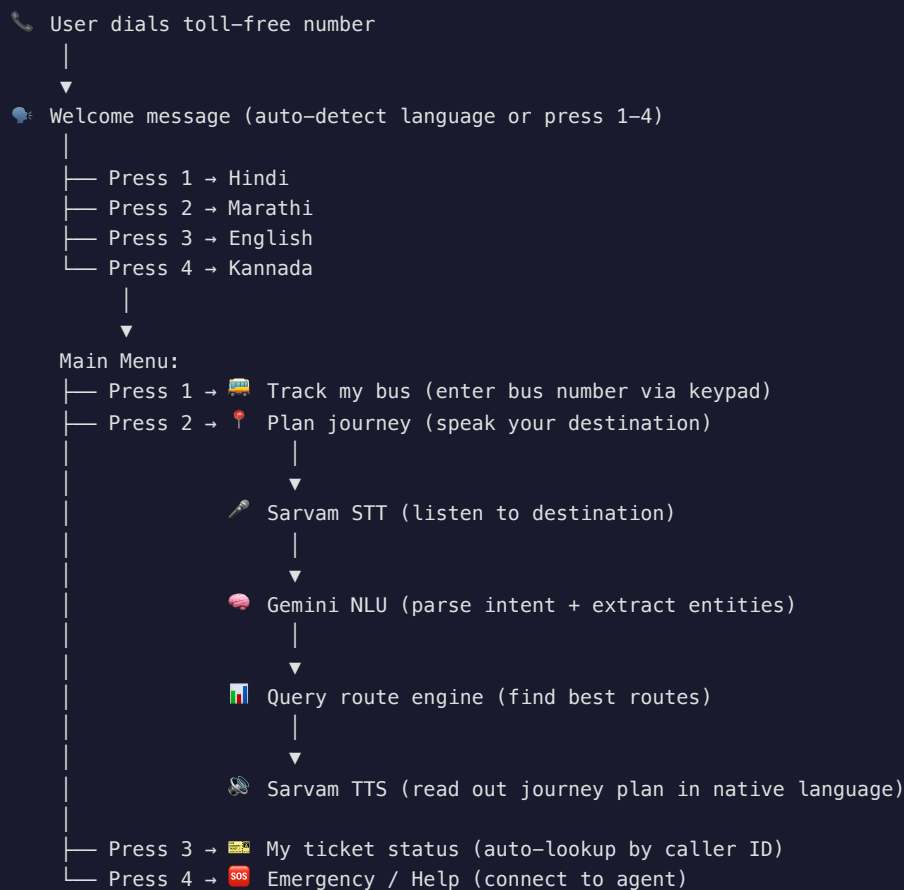
4.4 Connection Bus Management

When a journey requires changing buses:

1. **Pre-departure:** Show all connecting buses at the transfer stop, with real-time ETAs
2. **In-transit:** Continuously recalculate:
 - o "You'll reach Lonavala at ~2:30 PM"
 - o "Bus to Mumbai departs at 2:45 PM ( On time, 15 min buffer)"
 - o "Next alternative: 3:15 PM if you miss this one"
3. **Dynamic re-routing:** If the connecting bus is delayed/cancelled, auto-suggest alternatives
4. **Alert:** Push notification 5 min before reaching the transfer stop

4.5 IVR System for Elderly / Feature Phone Users

Call Flow



Conductor-Assisted Flow (Feature Phone Boarding)

1. Conductor issues ticket via POS
2. POS sends ticket data to Firestore with passenger's phone number
3. Cloud Function creates a "journey record" linked to that phone number

4. When the elderly person calls the IVR, system looks up their active journey by caller ID
5. IVR tells them: *"You are on Bus MH12AB1234. Next stop: Lonavala in 45 minutes."*

4.6 🌙 Night Halt Alert System

Trigger Flow

```

Step 1: Conductor taps "HALT STOP" button on POS/dashboard
|
▼
Step 2: Cloud Function fires:
|— Push notification to all smartphone passengers on this bus
|— Set 20-minute countdown timer
|
▼
Step 3: After 15 minutes:
|— "Bus leaving in 5 min" push notification
|
▼
Step 4: After 18 minutes:
|— Automated phone call (Exotel) to feature-phone passengers
|   TTS: "Aapki bus 2 minute mein chal rahi hai, kripya wapas aayein"
|
▼
Step 5: Conductor sees headcount dashboard:
|— Total passengers: 42
|— Confirmed back: 38
|— ⚠️ Not confirmed: 4 (names + seat numbers highlighted in red)

```

4.7 🚔 Conductor Dashboard

Feature	Description
Passenger Manifest	Live list of all ticketed passengers with phone numbers
Headcount Tool	One-tap check-in / check-out for halt stops
Halt Alert Trigger	Button to initiate halt-stop countdown alerts
Route Progress	Visual route progress bar with upcoming stops & ETAs
Delayed Bus Flag	Flag bus as delayed (auto-notifies connecting passengers)

5. Complete Data Model

5.1 Firestore Collections

```

Firestore
├── 👤 users/{userId}
│   ├── name, phone, language, preferredRoutes
│   └── activeJourney: { busId, boardStop, alightStop, status }
├── 🚌 routes/{routeId}
│   ├── name, operator, type (ordinary/express/sleeper)
│   ├── stops: [{ id, name, lat, lng, seq, avgDwell }]
│   ├── schedule: { frequency, firstBus, lastBus }
│   └── fare: { perKm, minimum }
├── 📍 stops/{stopId}
│   ├── name, lat, lng, amenities, district
│   └── connectedRoutes: [routeId1, routeId2, ...]
├── 🎫 tickets/{ticketId}
│   ├── passengerId, passengerPhone, busId, routeId
│   ├── boardStop, alightStop, fare, issuedAt
│   └── status: "active" | "completed" | "cancelled"
├── 📅 journeyPlans/{planId}
│   ├── userId, createdAt
│   ├── legs: [{ routeId, busId, boardStop, alightStop, etaBoard, etaAlight }]
│   └── status: "planned" | "in_progress" | "completed"
├── 📢 haltAlerts/{alertId}
│   ├── busId, routeId, stopName, triggeredAt
│   ├── resumeAt, conductorId
│   └── confirmations: { userId1: true, userId2: false }

```

5.2 Firebase RTDB (Real-Time Only)

```

🔥 RTDB
├── buses/{busId}
│   ├── lat, lng, speed, heading
│   ├── routeId, nextStopId, etaNextStop
│   ├── status, lastUpdated
│   └── passengerCount

```

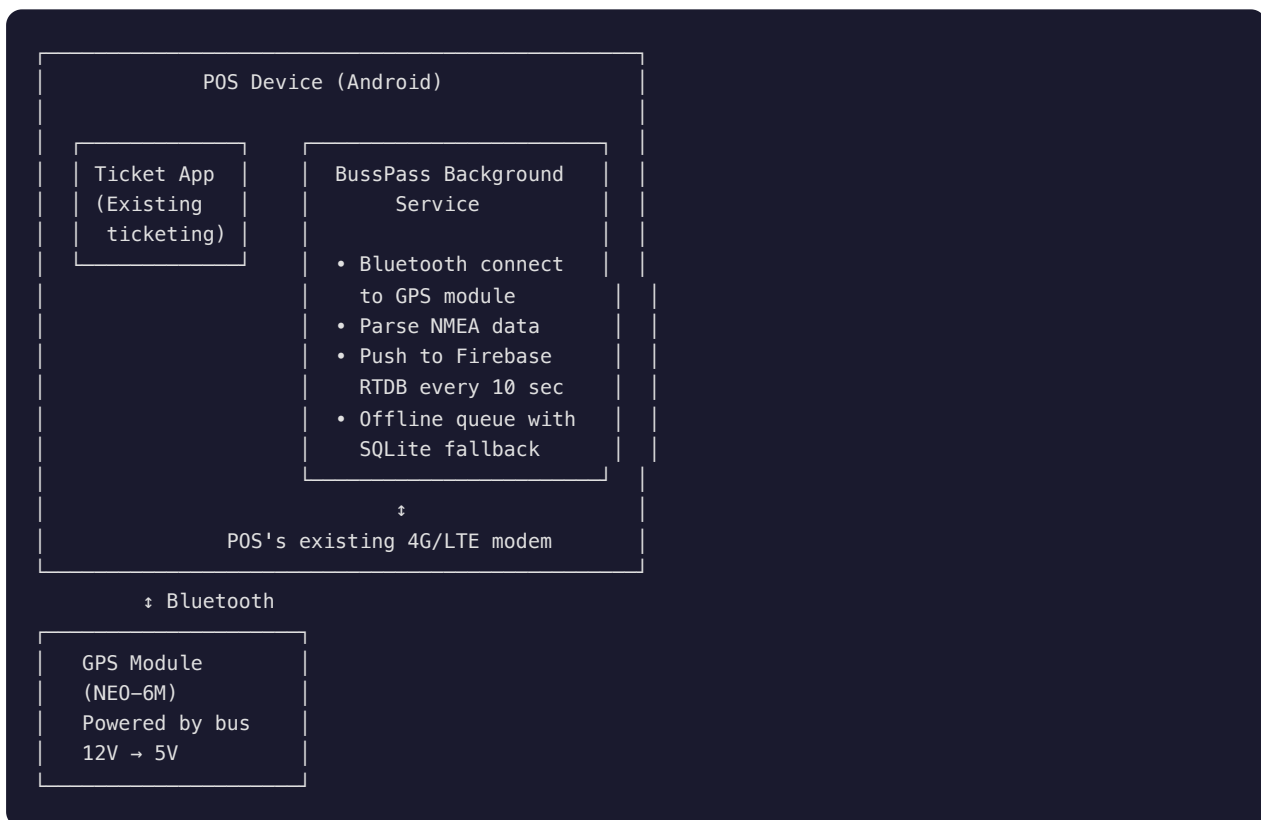
6. Cloud Functions API Design

Function	Trigger	Description
onBusLocationUpdate	RTDB write on <code>buses/{busId}</code>	Calculate ETA to next stops, update journey progress
planJourney	HTTPS callable	Takes src/dest/time → returns optimal routes

<code>detectTransit</code>	Scheduled (every 30s per active user)	Cross-reference user location with bus locations
<code>triggerHaltAlert</code>	HTTPS callable (conductor)	Send push notifications + schedule phone calls
<code>ivrWebhook</code>	HTTPS (Exotel callback)	Process IVR inputs, query routes, return TTS response
<code>ivrSttProcess</code>	HTTPS	Receive audio → Sarvam STT → Gemini NLU → intent
<code>linkTicketToPhone</code>	Firestore <code>tickets/{id}</code> create	Auto-create journey record for feature phone users
<code>connectionMonitor</code>	Pub/Sub (every 60s)	Check connecting bus availability for transit users

7. GPS + POS Integration Detail

7.1 POS Background Service Architecture



Key Advantage: The per-bus data consumption is only **~1.15 MB/day** — trivial on the POS device's existing data plan. The hardware components are widely available, low-power, and easy to install.

8. Security Architecture

Layer	Measure
Firestore Auth	Phone OTP authentication for all users
RTDB Rules	POS devices = write (service accounts); Users = read-only
Firestore Rules	Users read own journeys only; Conductors write own bus data
API Security	Cloud Functions behind Firebase App Check (device attestation)
POS Auth	Each POS gets unique service account key, rotated quarterly
IVR	Caller ID verification + OTP for sensitive operations
Data Privacy	Location data auto-deleted after 24h; anonymized for analytics
Compliance	DPDP Act 2023 compliant — no permanent location storage

9. Prototype Scope — SIH 36-Hour Build

⚠ For SIH, we build a working MVP, not a production system. The following is scoped for the 36-hour hackathon with a 6-member team.

9.1 Must-Have Features (Demo-Ready)

#	Feature	Priority	Est. Hours	Owner
1	Flutter app with Google Maps + live bus markers from RTDB	P0	6h	Dev 1
2	Simulated GPS data feed (script pushing fake bus locations)	P0	2h	Dev 2
3	Multi-bus journey planner with 2-3 demo routes	P0	6h	Dev 2
4	Smart transit detection (match user location to bus)	P0	4h	Dev 3
5	Night halt alert (push notification + phone call demo)	P0	3h	Dev 3
6	IVR demo with Exotel (DTMF menu + STT in Hindi/Marathi)	P0	5h	Dev 4
7	Conductor dashboard (web) with headcount	P0	4h	Dev 5
8	Connection bus management UI	P1	3h	Dev 5
9	Beautiful UI/UX with animations + pitch deck	P0	3h	Dev 6

Total: ~36 hours (6 team members × 6 hours each)

9.2 Nice-to-Have (If Time Permits)

Feature	Notes
Physical GPS hardware demo	If GPS module available — very impressive to judges
Offline mode with Hive caching	For low-connectivity demo scenario
Multi-language IVR (4 languages)	Start with Hindi + English minimum
Analytics dashboard	Show bus utilization, delay patterns

10. Project Structure

```

busspass/
├─ android/ & ios/
├─ web/                # Conductor dashboard
├─ lib/
│  ├─ main.dart
│  ├─ app.dart          # MaterialApp, routing, theme
│  ├─ core/
│  │  ├─ constants/     # Colors, strings, API keys
│  │  ├─ theme/          # App theme, typography
│  │  ├─ utils/          # Helpers, formatters
│  │  └─ services/
│  │     ├─ location_service.dart
│  │     ├─ notification_service.dart
│  │     └─ transit_detection_service.dart
│  ├─ data/
│  │  ├─ models/         # Bus, Route, Stop, Ticket, Journey
│  │  ├─ repositories/   # FirebaseRepo, LocationRepo
│  │  └─ providers/      # Riverpod providers
│  ├─ features/
│  │  ├─ home/           # Home screen with map
│  │  ├─ tracking/       # Live bus tracking screen
│  │  ├─ journey_planner/ # Source → Dest planner
│  │  ├─ transit/        # In-transit journey view
│  │  ├─ connections/    # Connecting bus info
│  │  ├─ alerts/         # Halt stop alerts
│  │  ├─ conductor/      # Conductor dashboard
│  │  └─ onboarding/     # Auth + onboarding
│  └─ widgets/           # Shared UI components
├─ functions/           # Firebase Cloud Functions
├─ src/
│  ├─ eta.ts
│  ├─ journey-planner.ts
│  ├─ halt-alerts.ts
│  ├─ ivr-webhook.ts
│  ├─ transit-detection.ts
│  └─ connection-monitor.ts
└─ scripts/

```

```

├── simulate_gps.py      # Simulate bus GPS for demo
├── seed_routes.py       # Seed Firestore with demo data
└── pubspec.yaml

```

11. Sprint Plan (Pre-SIH 4-Week Preparation)

Phase 1: Foundation (Week 1)

- ☐ Set up Flutter project with clean architecture (Riverpod)
- ☐ Set up Firebase project (RTDB + Firestore + Auth + Functions)
- ☐ Design app theme, design system, custom widgets
- ☐ Implement Firebase Auth (Phone OTP)
- ☐ Create all data models (Bus, Route, Stop, Ticket, Journey)
- ☐ Seed Firestore with 5-10 demo routes (MSRTC Pune-Mumbai corridor)

Phase 2: Core Features (Week 2)

- ☐ Build real-time map with live bus markers (RTDB → Google Maps)
- ☐ Build GPS simulator script (Python — moves bus along route polyline)
- ☐ Implement journey planner algorithm (graph-based Dijkstra)
- ☐ Build journey planner UI (source/dest input → route option cards)
- ☐ Implement smart transit detection background service
- ☐ Build in-transit journey view (progress bar, ETA, next stop alert)

Phase 3: IVR + Conductor (Week 3)

- ☐ Set up Exotel account + configure IVR flow (DTMF menus)
- ☐ Integrate Sarvam AI STT/TTS for Hindi + English
- ☐ Build IVR webhook Cloud Functions
- ☐ Build conductor dashboard (Flutter Web)
- ☐ Implement night halt alert system (push + phone call)
- ☐ Implement connection bus monitoring logic

Phase 4: Polish + Demo Prep (Week 4)

- ☐ End-to-end integration testing (all features working together)

- ☐ UI polish — animations, transitions, error handling, edge cases
- ☐ Build and rehearse demo script (8-minute time limit)
- ☐ Prepare hardware demo (if GPS module procured)
- ☐ Create pitch deck (PPT/Google Slides)
- ☐ Prepare and rehearse judge Q&A answers

12. Scalability & Production Roadmap

Metric	Prototype (SIH)	Production (Post-SIH)
Buses tracked	10 (simulated)	50,000+
Concurrent users	100	10M+
RTDB architecture	Single instance	Sharded by state/region
Journey planner	In-memory graph	Redis-cached + pre-computed
IVR capacity	Exotel sandbox	Multi-region redundant
GPS update interval	10 sec (fixed)	Adaptive (5s city, 30s highway)
Data retention	24 hours	90 days (cold storage)

13. Judge Q&A Preparation

⚠ CRITICAL SECTION — Every team member must memorize these answers.

Technical Feasibility

Q: How does GPS work if the POS device doesn't have GPS capabilities?

The GPS module is an external hardware unit (NEO-6M) connected to the POS via Bluetooth. It uses the bus's 12V power supply. The POS only acts as a data relay using its existing internet connection. No GPS capability is needed on the POS itself.

Q: What if the POS loses internet connectivity?

The POS background app queues location updates locally using SQLite. When connectivity resumes, it batch-uploads all queued updates. Firebase RTDB handles offline sync natively. The app shows "Last updated X minutes ago" to users.

Q: How do you handle the latency of real-time tracking?

GPS update every 10s → POS pushes to RTDB (200ms) → ~~Firestore propagates to clients (100ms)~~. Total end-to-end latency: **under 2 seconds**. For a bus at 60 km/h, 2s lag = 33 meters — well within acceptable UX.

Q: Can your system work in areas with poor connectivity (ghats, rural)?

Three layers of resilience: (1) POS queues data offline and syncs when connected, (2) Flutter app caches last known positions via Hive, (3) IVR is PSTN-based — works even with 2G signal where data fails.

Innovation & Novelty

Q: How is this different from the existing "Aapli ST" app?

Aapli ST is GPS tracking only. BussPass adds: (1) Multi-bus journey planning with real-time connections, (2) IVR for 300M+ feature phone users, (3) Smart transit detection, (4) Night halt alerts, (5) Conductor headcount tools. We solve the **accessibility gap** that no existing solution addresses.

Q: Why not just use the driver's phone for GPS?

Drivers change shifts, forget to keep apps running, or phones die. Our GPS module is **hardwired to the bus battery** and paired with the conductor's POS (always on during duty). This gives **100% uptime** without human dependency.

Scalability

Q: Can this scale to all of India's 1.5 lakh state transport buses?

Firebase RTDB can be sharded by state/region. Each shard handles ~20,000 buses easily. Cloud Functions auto-scale horizontally. The GPS hardware is simple, mass-producible, and uses existing POS connectivity — so deployment scales linearly without new infrastructure.

Q: What about data privacy concerns?

User location is processed in-memory only for transit detection, never stored permanently. Bus locations are public data. All personal data encrypted at rest. We comply with India's DPDP Act 2023. Location data auto-purged after 24

hours.

Impact

Q: How does the IVR help elderly users who can't read?

The IVR speaks in their native language. Example: An elderly Marathi speaker calls, presses 2 for Marathi, says "मला पुण्याहून मुंबईला जायचं आहे". Sarvam AI transcribes → Gemini understands → system finds routes → TTS reads out the journey plan in Marathi. The experience is **conversational**, not menu-driven.

Q: What if the elderly person doesn't know the bus number?

The conductor links the passenger's phone number to the ticket during boarding. After that, the IVR automatically identifies their active journey by caller ID. They call and hear: "You are on Bus 1234. Next stop: Lonavala in 45 minutes."

Q: How do you handle the night halt problem?

Conductor triggers halt alert → smartphones get push notification → feature phones get automated call in their language → conductor sees real-time headcount dashboard → after 15 min, final "bus leaving" alert fires. This eliminates passengers being stranded.

Deployment Model

Q: How would this be deployed in practice?

Primary model is **B2G (Business-to-Government)** — state transport corporations adopt the platform. GPS hardware is installed during routine bus maintenance. The POS background app is pushed as an OTA update. The passenger app is published on Play Store / App Store. IVR number is advertised at bus stands and on tickets.

14. Demo Script (SIH 8-Minute Presentation)

Time	Activity	What to Show
0:00 - 2:00	Problem Statement	Stats (70M passengers/day), pain points, gap analysis
2:00 - 5:00	Live Demo	

	1. Open app	Live map with 3-4 buses moving in real-time
	2. Plan journey	"Pune to Mumbai" → multi-bus route with connections
	3. Board bus	Transit detection → "You're on Bus MH12-AB-1234"
	4. Connection alert	"Connecting bus at Lonavala departs in 20 min"
	5. Halt stop	Conductor triggers halt → push notification appears
	6. IVR demo	Call number live → navigate in Hindi → get journey info
5:00 - 7:00	Architecture	System diagram, GPS+POS innovation, IVR flow
7:00 - 8:00	Impact & Scale	National scalability, social inclusion, deployment strategy

15. Verification Plan

Automated Tests

```
flutter test                                # Unit tests
cd functions && npm test                     # Cloud Function tests
flutter test integration_test/journey_flow.dart # E2E integration
```

Manual Verification

- Live demo with simulated GPS data on physical device
- IVR call test with Exotel sandbox number
- Multi-device test (passenger app + conductor dashboard simultaneously)
- Offline mode test (airplane mode → reconnect → data sync)
- Night halt alert end-to-end flow (push + phone call)

BussPass — Built with ❤️ for 70 Million Daily Passengers

Smart India Hackathon 2026

This document is confidential and intended for team use only.